

Report Review: Real-Time Machine Learning-Based Threat Detection for ISP Network Traffic

Reviewer: Xiaoyan Xiong

March 1, 2026

1 Project Workflow Overview

The goal of this project is to analyze network traffic data using a combination of deep learning and machine learning techniques to identify malicious traffic and classify it. The entire process involves several steps including data collection, processing, embedding generation, classification, and storing the results, as well as retraining the model. Below is the overall workflow of the project:

1.1 Data Collection and Preprocessing

The initial data comes from PCAP files (Packet Capture), which record network packets. In order to extract useful information from raw network traffic data, the data is first processed using Suricata:

- Suricata is an open-source network intrusion detection system (IDS) that captures and analyzes network traffic. Suricata parses the header and payload of each packet and converts this information into structured JSON format. The JSON data includes source IP, destination IP, protocol, port number, TTL, ToS, etc., as well as the payload (content) part.

- This transformed JSON data is stored in Redis, a high-performance key-value store that is well-suited for storing and transmitting real-time data streams. The data is stored in Redis as streams, with each packet’s information represented as a message in the stream.

1.2 Data Reading and Embedding Generation

Once the data is stored in Redis, the next step is to read the data from the Redis stream. The process is as follows:

- An asynchronous Redis client is used to read data from the Redis stream (`suricata:packets`). The header and payload of each packet are extracted and passed to the model for processing.
- These header and payload parts are processed through the BigBird Transformer model to generate an embedding vector. BigBird is a transformer model designed to handle long text sequences and effectively encodes text data into fixed-length embedding vectors. In this process, both the header and payload are processed separately, and each part generates a 32-dimensional embedding vector. Finally, these two embeddings are concatenated to form a 64-dimensional vector.

1.3 Classification Using Machine Learning Model

The Random Forest Classifier is used to classify the generated embedding vectors. This model has already been trained and saved using scikit-learn’s joblib tool. The classifier’s task is to determine whether the network traffic is normal or malicious based on the embedding vectors.

- The resulting embedding vectors and their classification labels (malicious or normal) are then stored in the PostgreSQL database. The database is used for long-term storage of prediction results and embedding vectors, enabling subsequent analysis, visualization, and further model training.

1.4 Data Storage and Model Retraining (Active Learning)

Once the data is stored in the database, Active Learning is applied to improve the model. Specifically, the classification results and embedding vectors are used for further model training to improve its accuracy and robustness.

- New data is continuously read from Redis, classified by the existing model, and stored in the database. This newly stored data is then used to update and retrain the model. This process allows the model to continuously improve and adapt to new patterns of malicious traffic.

1.5 User Interaction and Frontend Visualization

Finally, all classification results are displayed to the user via the Streamlit framework. Streamlit is a tool for building interactive web applications. On the frontend page, users can see the prediction results for each packet, including source IP, destination IP, and the predicted label (malicious or normal).

- Users can also manually update the labels on the page or send the prediction results to the database for further model training.

1.6 Technologies and Tools

- Suricata: Used for capturing and parsing network traffic data, converting it into JSON format.
- Redis: Used for real-time storage and transmission of stream data, supporting efficient data reading and stream operations.
- BigBird Transformer: Used for converting header and payload into embedding vectors, processing long text sequences.
- Random Forest Classifier: Used to classify the generated embedding vectors, determining whether the network traffic is malicious or normal.
- PostgreSQL: Used to store the classification results, embedding vectors, and other metadata for persistent storage.

- Streamlit: Used for displaying prediction results and providing an interactive user interface.
- Active Learning: Applied to continuously improve the model by incorporating new data into the training process.

1.7 Summary

This project combines deep learning (BigBird) and traditional machine learning (Random Forest) methods to provide an efficient framework for analyzing network traffic. Initially, network traffic data is captured by Suricata, converted into JSON format, and stored in Redis streams. Then, BigBird Transformer generates embedding vectors for both the header and payload, which are classified by a Random Forest Classifier. The results and embedding vectors are stored in a PostgreSQL database for further analysis and model updates through Active Learning. This process ensures that the model continuously improves over time and adapts to new malicious traffic patterns.

the project workflow being summarized after discussions with the previous intern and reviewing parts of the code, some parts may involve misunderstandings or assumptions based on those interactions.

2 Report Review

2.1 Overview and Clarity of the Paper

The paper presents a comprehensive approach to detecting malicious network traffic using a combination of deep learning and machine learning models, specifically BigBird Transformer and Random Forest. The methodology is sound and the proposed solution is appropriate for the problem at hand. However, there are several areas where clarity could be improved, particularly in terms of explaining some technical details and justifying certain choices, which would help readers with limited background in the domain fully understand the approach.

2.2 Problem Definition and Motivation

The problem is well-defined, and the motivation for using machine learning techniques, particularly BigBird Transformer, is justified. However, the choice of BigBird over other models like BERT or GPT is not clearly explained. The paper would benefit from a more thorough discussion of why BigBird was selected, particularly in the context of network traffic data and the model's ability to handle long sequences.

2.3 Methodology

The methodology is generally sound, leveraging Suricata for packet capture, BigBird for embedding generation, and Random Forest for classification. However, the paper could be improved by providing more detail on:

- How the tokenizer was trained, especially the specifics of the vocab size and training corpus.
- The choice of machine learning models and the rationale behind selecting them.
- The Active Learning process and how expert feedback is incorporated.

Additionally, it would be helpful to clarify whether Suricata is using rule-based detection or AI-based detection to generate labeled data, as this impacts the accuracy and bias of the labels.

2.4 Experimental Setup and Evaluation

The experimental results demonstrate the effectiveness of the approach, particularly the use of Random Forest for classification. However, the evaluation could benefit from the following improvements:

- Expanding the evaluation metrics to include precision, recall, and F1-score.
- Using k-fold cross-validation instead of a simple train-test split to account for the small dataset size.
- A more detailed discussion of the real-time performance of the system, especially regarding the training time and inference time of BigBird.

2.5 Active Learning

The paper briefly mentions Active Learning but does not explain how the feedback loop is implemented. It would be helpful to expand on how expert feedback is incorporated into the model and how it helps improve the model over time.

2.6 Related Work and Future Work

The related work section could benefit from a more thorough comparison with traditional IDS/IPS systems and other machine learning-based methods for traffic analysis. The future work section mentions migrating to AWS and improving the UI but does not discuss in-depth how real-time processing and scalability will be addressed.

2.7 Conclusion

The paper presents a promising approach for real-time network traffic analysis using machine learning, with a particular focus on BigBird Transformer and Random Forest. However, there are areas for improvement, such as the explanation of model choices, the evaluation process, and the Active Learning loop. The methodology can be strengthened by providing more detailed experimental results and further justifying the design choices.

3 Identified Issues and Suggestions for Improvement

3.1 Dataset Size and BigBird Model

The dataset used in the paper consists of only 400 samples. Using such a small dataset, particularly with a large model like BigBird Transformer, may lead to overfitting and issues with dimensionality (curse of dimensionality). A smaller model or more data augmentation techniques should be considered to mitigate this issue.

3.2 Insufficient Information for Reproducibility

While some details of the models are provided, there is a lack of sufficient information for reproducing the experiments. For instance, the hyperparameters, tokenizer training details, and other setup information are missing. These details are crucial for ensuring that the experiment can be replicated.

3.3 Justification for Choosing BigBird Transformer

The paper does not provide enough justification for choosing the BigBird Transformer model over other potential models like BERT or GPT. More discussion on why BigBird is well-suited for this problem, particularly for handling long sequences of network traffic data, would strengthen the methodology.

3.4 k-fold Cross-Validation

With the small dataset size of 400 samples, the use of k-fold cross-validation would be a better approach than a simple train-test split. This would help in providing a more reliable estimate of the model's performance.

3.5 Tokenizer Training and Coverage

The tokenizer was trained on custom data, but the paper does not explain how it handles evolving network information. It is important to discuss how often the tokenizer should be updated and whether it is capable of covering new network patterns as they emerge.

3.6 Labeled Data Generation

The paper mentions that Suricata generates the labeled data, but it is unclear whether this is based on rule-based detection or AI-based detection. Understanding this is crucial, as it can affect the accuracy and bias of the labels.

3.7 Real-Time Processing Considerations

The title mentions Real-Time detection, but there is no discussion of the time required for training and inference. Given that BigBird is a large transformer model, a discussion on the time required for training and real-time prediction would help evaluate the practicality of this approach.

3.8 Active Learning Optimization

While Active Learning is mentioned, the paper does not provide enough details on how the feedback loop works. The inclusion of expert validation would be helpful to ensure that the model continues to improve over time.

3.9 Lack of Contextual Information in Packet-Level Detection

The use of packet-level detection might be limited in that it does not capture the full context of network traffic. It would be helpful to discuss how contextual information could be incorporated to improve detection accuracy.

3.10 Data Preprocessing Issues

There is no mention of data preprocessing, such as handling empty values or removing irrelevant data. Proper data cleaning is crucial for improving model performance.